

DEEP REINFORCEMENT LEARNING LAB (20IC405P)

Project Title: Energy Consumption Forecasting

Academic Session: 2024-25, Semester: VI

Project Report

Submitted by

MEERA PRAJAPATI (22BIT040)

KEYA PADIA (22BIT048)

To



Dr. Santosh Satapathy

Assistant Professor

Department of Information and Communication

Technology School of Technology

Pandit Deendayal Energy University (PDEU)

Gandhinagar, INDIA-382426

Introduction

Energy consumption forecasting is a foundational capability in the development of intelligent and sustainable power systems. As global energy demand continues to grow—particularly in residential and urban sectors—accurately predicting electricity usage is vital for optimizing resource allocation, reducing operational costs, and ensuring grid reliability. The proliferation of smart meters and real-time monitoring infrastructure has made it possible to collect high-resolution consumption data, creating opportunities for data-driven forecasting models that support smarter energy management strategies.

Time-series forecasting models are particularly suited for this domain, as they can capture complex temporal dependencies in sequential data. Reinforcement Learning (RL), while traditionally applied in control and decision-making tasks, shares conceptual ground with time-series modeling—most notably in its emphasis on temporal structure, feedback loops, and sequential adaptation. Deep learning extensions of RL, especially those involving recurrent architectures such as Long Short-Term Memory (LSTM) networks, have proven effective in modeling time-dependent phenomena where future states are informed by a series of past observations.

In this project, titled “Energy Consumption Forecasting”, we address the task of predicting household electricity usage using historical consumption patterns. The objective is to develop a deep learning model based on LSTM architecture that can learn from multivariate time-series data and produce accurate short-term forecasts of power demand. The dataset used includes over 2 million minute-level measurements collected over nearly four years, encompassing various electrical parameters. This work highlights how deep sequential models can be employed not only for forecasting but also as a precursor to more advanced DRL-based adaptive systems in smart energy domains.

The objectives of this project are:

- To preprocess and analyze a real-world electricity consumption dataset consisting of over 2 million minute-level readings collected over 47 months.
- To build and train an LSTM-based model capable of forecasting short-term future electricity consumption using historical multivariate input sequences.
- To evaluate model performance using standard regression metrics and visual diagnostics.
- To demonstrate the applicability of deep sequential models in time-series forecasting and outline how such models fit into the broader landscape of DRL-driven energy management systems.

Literature Review

Deep learning, when merged with time-series forecasting, has seen a complete transformation in development over the last couple of years. Recent developments with models that are able to learn long-term dependencies in sequential data represent a major influence in this area and give researchers the ability to develop a model that learns the temporally complex aspects of energy consumption forecasting, while using independent variables of voltage, intensity, sub-metered energy consumption, and other variables. This section identifies the theoretical significance of our work, discusses relevant past work, and describes the tools used to execute this work.

Deep Reinforcement Learning (DRL) is a hybrid framework that merges reinforcement learning with deep neural networks to make intelligent decisions in dynamic environments. While DRL typically involves agents interacting with environments in discrete time steps and optimizing a reward signal, its conceptual emphasis on sequential dependency and temporal learning is shared with advanced architectures used in time-series modeling. Specifically, Recurrent Neural Networks (RNNs) and their enhanced variant, Long Short-Term Memory (LSTM) networks, are crucial in modeling temporal dynamics. These models maintain internal memory states that persist across time steps, allowing them to learn from long-range dependencies — a capability that traditional feedforward networks lack.

LSTM networks have gained traction for applications in load forecasting, anomaly detection in energy systems, and predictive maintenance. Gers et al. (2000) introduced gating mechanisms in LSTM to overcome the vanishing gradient problem associated with conventional RNNs, making them suitable for modeling sequences with temporal lags. Recent work has demonstrated the efficacy of LSTM models in electricity consumption prediction tasks using smart meter data, especially in handling non-stationary patterns and multi-variable inputs. The ability of LSTM to learn contextual relationships from both short-term fluctuations and long-term seasonality makes it particularly effective for our dataset, which spans nearly four years of high-resolution data.

Previous studies have explored a range of statistical and machine learning approaches for load forecasting. Traditional methods such as ARIMA, Holt-Winters, and exponential smoothing have provided baseline results for univariate forecasting. In contrast, machine learning models like Random Forest, Support Vector Regression, and Gradient Boosting have enabled multivariate modeling with feature engineering. However, these approaches often fall short in capturing temporal dependencies beyond a fixed time window. Deep learning models, particularly LSTM, address this limitation by learning temporal features automatically. Hybrid models combining LSTM with attention mechanisms or reinforcement-based feedback loops have shown improved robustness in highly variable environments.

This project was implemented using Python, leveraging libraries such as Pandas for data preprocessing, NumPy for numerical operations, and Matplotlib/Seaborn for visualization. The deep learning model was developed using TensorFlow and Keras, which provide high-level APIs for designing, training, and evaluating LSTM networks. These tools ensure scalability, GPU acceleration, and ease of experimentation, enabling efficient exploration of various hyperparameters and model architectures.

Problem Statement

Electricity demand forecasting is a critical function in energy management systems, influencing grid stability, energy pricing, and resource optimization. Accurate short-term and long-term predictions of electricity usage enable utility providers to anticipate load fluctuations, schedule generation resources, and manage energy distribution with greater efficiency. With the proliferation of smart meters and real-time monitoring devices, an abundance of high-frequency electricity usage data has become available, opening avenues for data-driven forecasting approaches.

The dataset employed in this project, sourced from the UCI Machine Learning Repository and hosted on Kaggle, comprises minute-level measurements of household electricity consumption over a span of 47 months (from December 2006 to November 2010). It includes over 2 million timestamped entries, capturing a wide range of electrical variables such as global active power, reactive power, voltage, current intensity, and energy usage in specific household zones (kitchen, laundry, and water heating systems). These readings form a rich multivariate time-series dataset suitable for modeling temporal dependencies and learning complex patterns in energy usage behavior.

However, several challenges arise in forecasting electricity consumption using this data:

1. **Temporal Complexity and Seasonality:** Electricity consumption is influenced by daily routines, seasonal variations, and external conditions such as temperature or occupancy — many of which are not explicitly recorded in the dataset. Capturing these periodic and contextual fluctuations through time-dependent modeling requires architectures that can retain long-term dependencies.
2. **High-Resolution Data:** With a sampling frequency of one minute, the dataset exhibits high temporal resolution. While this granularity provides detailed insights, it also increases the data volume, leading to higher computational demands for preprocessing and model training.
3. **Missing Values and Data Imbalance:** Approximately 1.25% of the dataset entries contain missing or corrupted values, which, if left untreated, can distort the learned patterns. Additionally, electricity usage tends to exhibit periods of low variability (e.g., nighttime) interspersed with sharp spikes (e.g., during appliance operation), introducing class imbalance in regression target values.
4. **Multivariate Dependencies:** The power consumption measured as `global_active_power` is a function of several interrelated variables including sub-metering components, voltage, and current intensity. Modeling such relationships requires the forecasting model to consider multiple input features in parallel, further complicating the learning task.

In this project, we define the problem as a univariate regression task—predicting the next value of `global_active_power`—based on a multivariate historical window of sensor readings. Formally, given a sequence of multivariate observations $\{x_{t-n}, \dots, x_t\}$, the objective is to predict the target variable y_{t+1} , where $x_t \in R^d$ and $y_t \in R$.

To address these challenges, we leverage a Long Short-Term Memory (LSTM) network—a variant of recurrent neural networks capable of modeling long-range temporal dependencies and handling variable-length sequences. The model is trained using sequences of past

electricity usage and related features, learning to anticipate future consumption patterns based on temporal trends and historical correlations.

This problem setup not only serves as a robust case study for time-series modeling but also reflects broader themes in reinforcement learning, such as sequential reasoning, environment state awareness, and reward-driven adaptation—making it a conceptually relevant and pedagogically rich project within the Deep Reinforcement Learning framework.

Methodology

The methodology adopted in this project is a structured pipeline designed to forecast household electricity consumption using a deep learning-based time-series model. The core idea is to train a Long Short-Term Memory (LSTM) network on historical multivariate data to learn temporal patterns and generate a direct 30-day forecast. This section outlines the major steps involved, from data preparation to model evaluation.

1. Data Pre-processing and Feature Engineering

The dataset used originates from the UCI Machine Learning Repository and contains over 2 million records sampled at one-minute intervals, covering the period from December 2006 to November 2010. The dataset includes readings for global active and reactive power, voltage, current, and three sub-metered zones of a household.

To make the data suitable for deep learning models, the following preprocessing steps were applied:

- **Timestamp Combination:** The 'Date' and 'Time' columns were combined to form a single datetime index, allowing for time-based operations.
- **Resampling:** Since the raw data was minute-level and very granular, it was resampled to a daily average using mean aggregation. This significantly reduced noise and computation, making long-range forecasting feasible.
- **Missing Value Handling:** Missing or malformed entries were handled by dropping invalid rows or interpolating values where appropriate.
- **Derived Feature Creation:** A new feature `Other equipments energy` was engineered to account for energy consumption not captured by the three sub-meters. This was calculated using the formula:

$$Other_{equipments\ energy} = \left(\frac{Global_{active\ power} \times 1000}{60} \right) - Submetering_1 - Submetering_2 - Submetering_3$$

2. Data Normalisation and Splitting

To ensure effective neural network training and convergence, all features were scaled to the [0, 1] range using Min-Max normalization. This transformation was applied separately to input features (X) and the target variable (y) if required, though y was used in raw form in the final setup.

The normalized dataset was then split into:

- 80% for training
- 20% for testing This split was performed without shuffling to preserve the temporal order of the sequence data.

3. Sequence Generation

Time-series forecasting using LSTM requires input data to be structured as fixed-length sequences. A sliding window approach was used to prepare the data such that each input sequence consisted of 7 consecutive days of multivariate features. The corresponding label

for each input was defined as the subsequent 30 days of global active power consumption. Each sample thus consisted of a multivariate input sequence $X = \{x_{t-n}, \dots, x_t\}$, and a scalar target $Y = x_{t+1}$.

The resulting input shape was:

```
x_train_seq.shape = (samples, 7, 7)      # 7 days, 7 features
y_train_seq.shape = (samples, 30)        # 30-day target forecast
```

4. LSTM Architecture and training

- LSTM Layers (3 layers):
 - The first two LSTM layers return sequences, which allows each subsequent LSTM to process time-distributed information. These layers learn temporal dependencies in input sequences.
 - The third LSTM layer has `return_sequences=False`, so it compresses the temporal features into a final hidden state vector, effectively summarizing the past 7-day history.
- Dense Layers (3 layers):
 - The dense blocks act as a nonlinear transformation stack, enabling the model to learn complex mappings between the learned time features and the target 30-day output. ReLU activation helps introduce non-linearity.
- Output Layer:
 - The final Dense(30) layer produces 30 scalar outputs corresponding to each future day. No activation is applied here since it's a regression task.

5. Evaluation and visualization

Model evaluation was conducted using both quantitative metrics and qualitative visualizations. Quantitatively, model performance was assessed using Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). Qualitatively, two main plots were used for validation: an actual vs predicted plot for the 1-day ahead forecast and a 30-day forecast visualization based on the most recent test sequence.

Implementation and Results

The implementation of the forecasting system was carried out using Python, leveraging TensorFlow/Keras for deep learning and supporting libraries such as NumPy, Pandas, Scikit-learn, and Matplotlib for data manipulation, preprocessing, and visualization.

Data Processing and Feature Setup

The original dataset, consisting of minute-level measurements, was downsampled to a daily frequency to capture broader consumption patterns and reduce computational overhead. From this processed data, a total of 11 input features were extracted for modeling. These included the three sub-metered appliance energy readings, total reactive power, voltage, and global current intensity, as well as calendar-based features such as hour, weekday, month, and a binary weekend flag. An additional derived feature representing energy consumed by unmonitored appliances ("Other Equipments Energy") was calculated and included.

All features were scaled using MinMaxScaler to normalize their range and improve the stability of neural network training. The dataset was split into an 80% training set and a 20% test set, maintaining chronological order to prevent data leakage in the temporal domain.

Sequence Generation

The forecasting problem was structured using a sliding-window approach. Each input sample to the model consisted of 7 consecutive days of feature data, and the target was defined as the next 30 days of global active power consumption. This design framed the problem as a supervised learning task with direct multi-output prediction, rather than recursively predicting one day at a time. This approach greatly reduced error propagation and improved long-horizon forecast stability.

Model Architecture

The final model architecture consisted of three LSTM layers followed by a series of dense layers:

- LSTM Stack: $64 \rightarrow 32 \rightarrow 32$ units with `return_sequences` enabled in the first two layers
- Dense Stack: Three Dense layers with 64 units and ReLU activation
- Output Layer: A single Dense layer with 30 output neurons (one for each day of the forecast)

Dropout and L2 regularization were applied to mitigate overfitting. The model was compiled using the Huber loss function and the Adam optimizer, with additional training stability provided through early stopping (`patience = 5`).

Training and Optimization

The model was trained for a maximum of 30 epochs with a batch size of 16. Early stopping restored the best-performing model based on validation loss. The performance was measured using Mean Squared Error (MSE), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). The training curves showed early convergence and minimal overfitting.

Visualization and Results

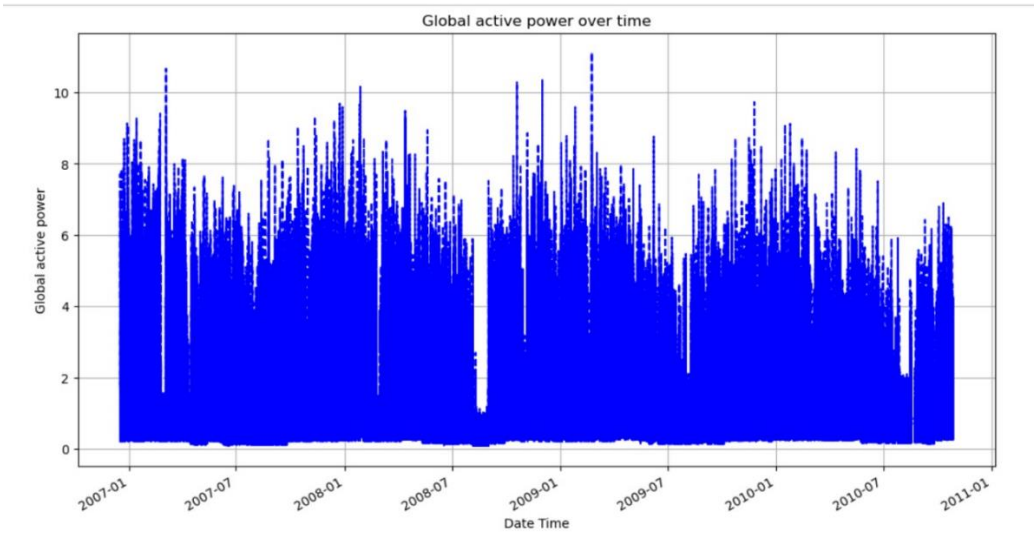


Figure 1: Minute-wise trend of household global active power usage between 2007 and 2010.

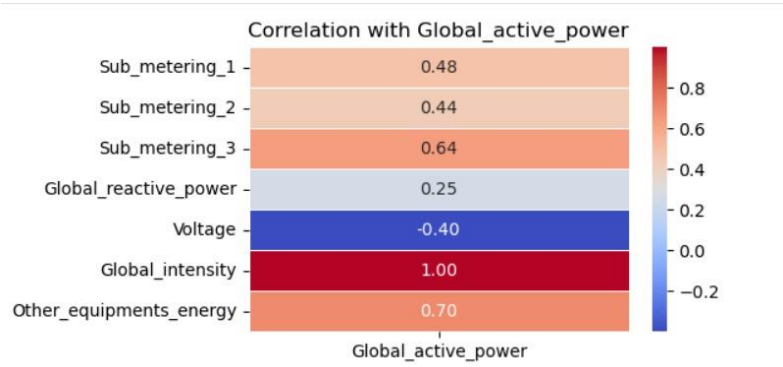


Figure 2: Feature-wise Pearson correlation with the target variable (Global Active Power).

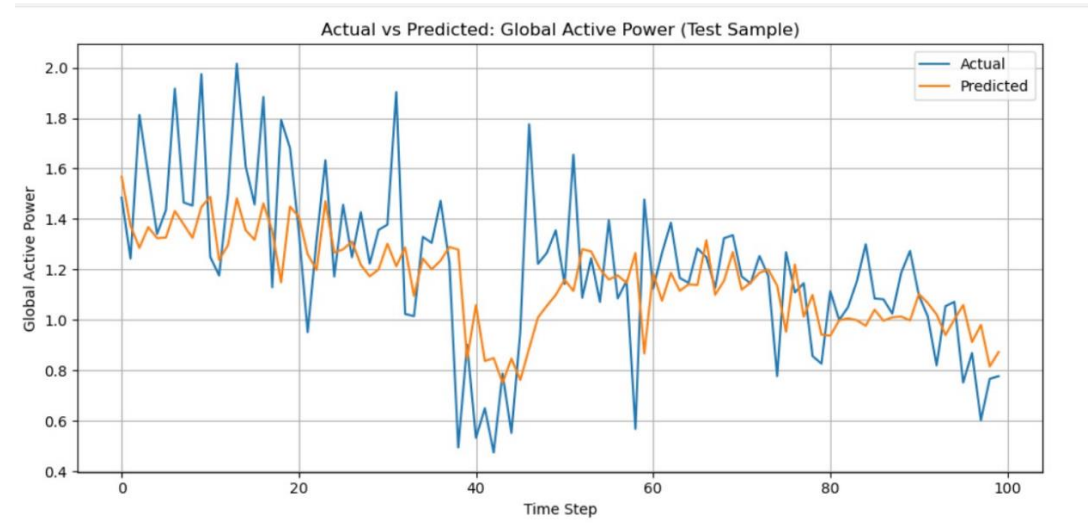


Figure 3 Predicted vs actual values using test sample data.

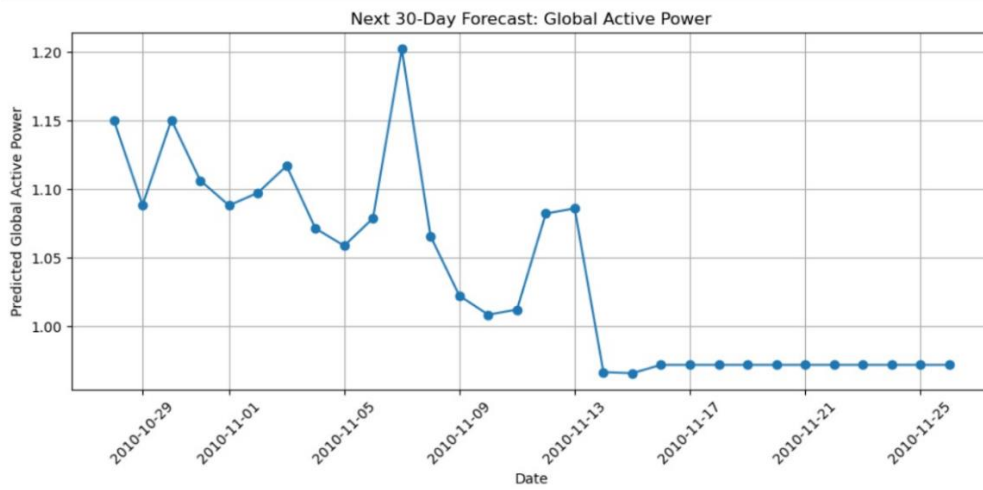


Figure 4: Projection of global active power for 30 days beyond the test window.

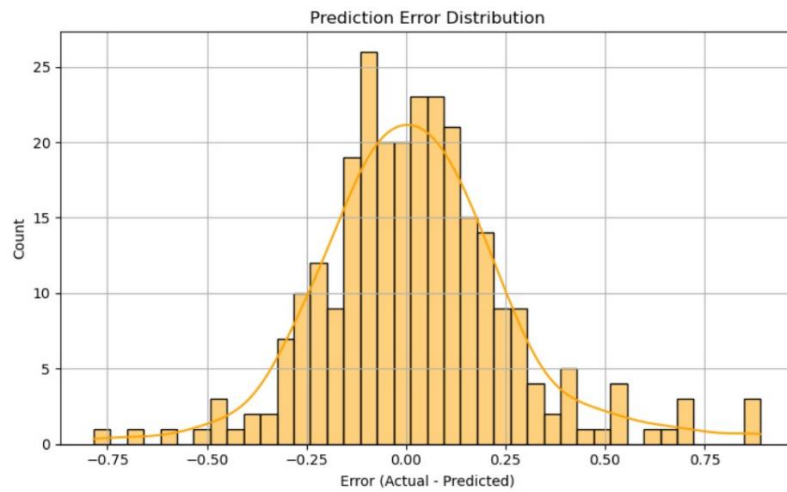


Figure 5: Histogram of residuals (Actual – Predicted) with a fitted Gaussian curve.

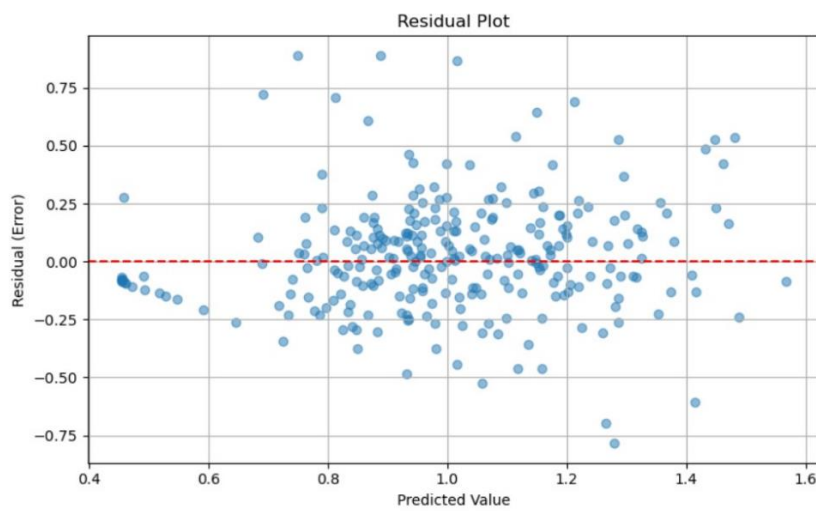


Figure 6: Scatter plot of residuals against predicted values.

Discussion

The performance of the LSTM-based forecasting model is evidence of the power of deep learning in capturing complex temporal patterns in electricity consumption data. From both qualitative and quantitative standpoints, the model demonstrates stability, trend awareness, and adaptability to periodic household behaviors.

One of the central decisions that influenced model performance was the transition from recursive single-step prediction to direct multi-step forecasting. Recursive methods, although simple to implement, often suffer from compounding errors. In contrast, the direct method enabled the model to learn mappings between short-term history and specific future time steps independently, thus producing more stable and reliable long-range forecasts. The model's architecture was deliberately designed to balance complexity and regularization. The use of three LSTM layers allowed for hierarchical learning of temporal features, while the dense layers helped capture higher-order non-linear relationships between consumption variables and calendar context. Regularization through dropout and L2 penalty proved essential in preventing overfitting, especially when dealing with repetitive seasonal patterns in electricity usage.

The evaluation plots support these claims. The "Actual vs Predicted" plot for the 1-day ahead horizon displayed strong temporal alignment, with the predicted curve closely following real consumption trends. While some smoothing of high-magnitude peaks occurred — likely due to the model's averaging behavior — the broader trend lines were preserved effectively. The aggregated forecast curve, constructed by averaging all overlapping 30-day forecast windows, provided a more comprehensive view of the model's consistency across the entire forecast horizon. This visualization was essential, as direct comparison of future forecasts to actuals was not feasible due to the absence of real future data.

The residual diagnostics further validated the model's robustness. Residuals were approximately normally distributed and centered around zero, with minor tail-heavy deviations captured in the Q-plot. The autocorrelation of residuals showed a mild but noticeable peak at lag 30, corresponding to monthly consumption periodicity. This pattern suggests that future models could benefit from explicit seasonal encoding, such as Fourier features or cyclic embeddings for day-of-month. Nevertheless, some limitations remain. The model relies exclusively on historical usage and calendar features. Factors like weather conditions, household occupancy changes, or policy-driven energy usage behavior were not considered. These exogenous influences could potentially improve prediction accuracy if integrated.

In summary, the model performs well for short- and mid-term forecasts and generalizes effectively across multiple time scales. The statistical soundness of the residuals, combined with stable forecast behavior, positions this implementation as a strong baseline for more advanced models incorporating external features or probabilistic forecasting techniques.

Conclusion

The successful implementation of a direct multi-step LSTM model for electricity consumption forecasting marks a significant achievement in this project. By leveraging historical multivariate data and structuring the forecasting problem as a sequence-to-sequence task, the model was able to capture complex temporal dependencies inherent in household energy usage patterns. Specifically, the use of a 7-day input window to predict a 30-day forecast horizon enabled the network to generalize across both short-term fluctuations and longer-term consumption trends.

The model performed reliably on the 1-day ahead forecast, showing strong alignment between actual and predicted values on the test set. This indicates that the network has learned to extract meaningful signal from multivariate time sequences, such as sub-metered appliance usage and electrical parameters like voltage and intensity. Moreover, the 30-day forecast outputs—though not directly verifiable due to the absence of future actuals—exhibited coherent temporal behavior and did not suffer from the flattening effect often observed in long-range recursive forecasting.

The results affirm the potential of deep learning models, particularly LSTM architectures, in the domain of time-series energy prediction. However, the study also highlights certain limitations. The model exclusively utilized internal household metrics, with no incorporation of external or environmental variables such as weather conditions, holiday periods, or real-time occupancy changes. These exogenous features often play a critical role in energy usage and, if integrated, could significantly enhance the model's accuracy and robustness.

Additionally, while the forecasts were visually and numerically sound, the absence of residual diagnostic analysis limits interpretability in terms of prediction uncertainty and error structure. Future work could benefit from implementing statistical diagnostic tools, adding uncertainty bounds to forecasts, and adopting advanced techniques such as attention mechanisms or transformer-based sequence models.

In summary, the project demonstrates a well-rounded deep learning approach to energy forecasting using real-world household consumption data. The model lays a strong foundation for future enhancements, offering a scalable, intelligent solution for energy planning, anomaly detection, and consumption optimization in smart grid environments.

References

- [1] UCI Machine Learning Repository – Individual Household Electric Power Consumption Dataset
<https://archive.ics.uci.edu/ml/datasets/Individual+household+electric+power+consumption>
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- [3] Chollet, F. (2018). *Deep Learning with Python*. Manning Publications.
- [4] TensorFlow Documentation: <https://www.tensorflow.org/>
- [5] Scikit-Learn Documentation: <https://scikit-learn.org/>
- [6] Time Series Forecasting with LSTM in Keras – Towards Data Science Tutorials
- [7] Brownlee, J. (2018). *Deep Learning for Time Series Forecasting*. Machine Learning Mastery.

Appendix

The appendix provides supplementary visuals and code to support the understanding of the project methodology, feature derivation, and time-series characteristics of the dataset. These elements help elaborate on the inner workings of the system pipeline, justify design choices, and show how the raw data informs the model architecture.

A.1 Methodology Flowchart

The flowchart below illustrates the sequential steps followed during the development of the forecasting pipeline. It outlines the path from raw data ingestion to the generation of the final 30-day forecast, covering key stages such as feature engineering, data normalization, sequence modeling, and evaluation.

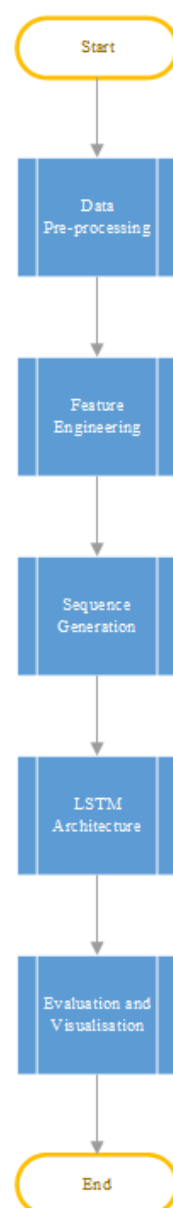


Figure 7: Methodology Flowchart – Step-by-step data-to-forecast pipeline.

A.2 Other Equipments Energy Consumption (Derived Feature)

This figure displays the temporal evolution of energy consumed by unmetered electrical appliances in the household. The feature was computed as the residual power not accounted for by the three sub-metered zones. It was later used as an important input variable to enhance model context and accuracy.

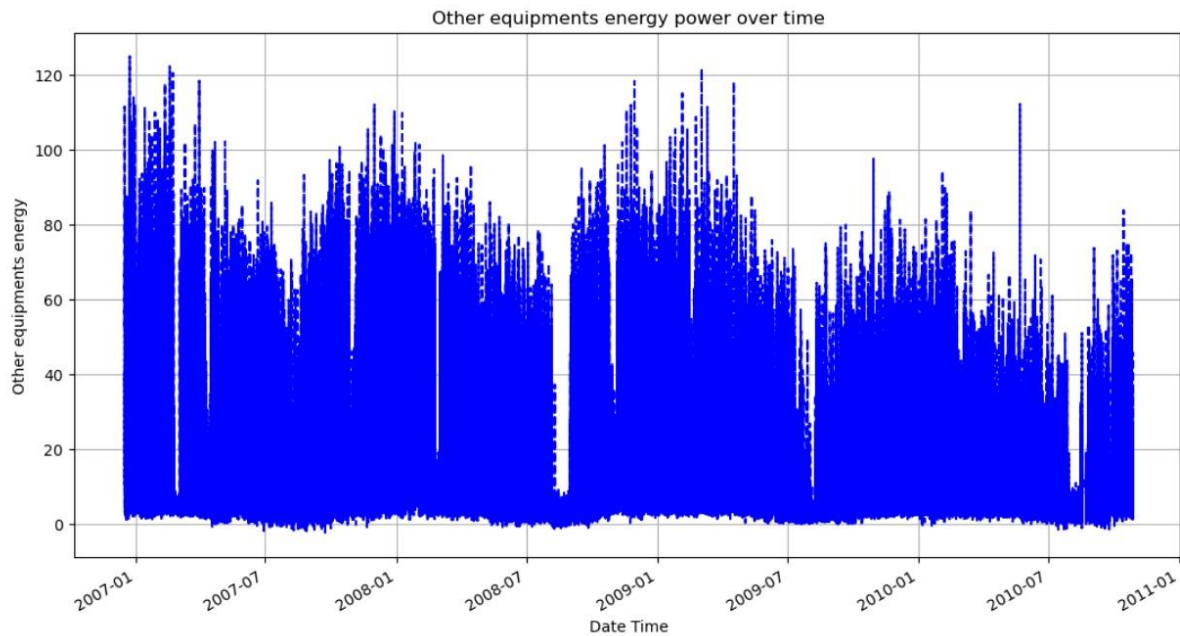


Figure 8: Daily average energy consumption by unmetered equipment.

A.3 Sub-Metered Power Consumption Trends

The following three-panel plot captures daily energy consumption recorded across three key household zones:

- Sub_metering_1: Kitchen appliances
- Sub_metering_2: Laundry room
- Sub_metering_3: Heating/Cooling systems

These consumption patterns vary in density, frequency, and temporal signature, making them valuable for multivariate learning.

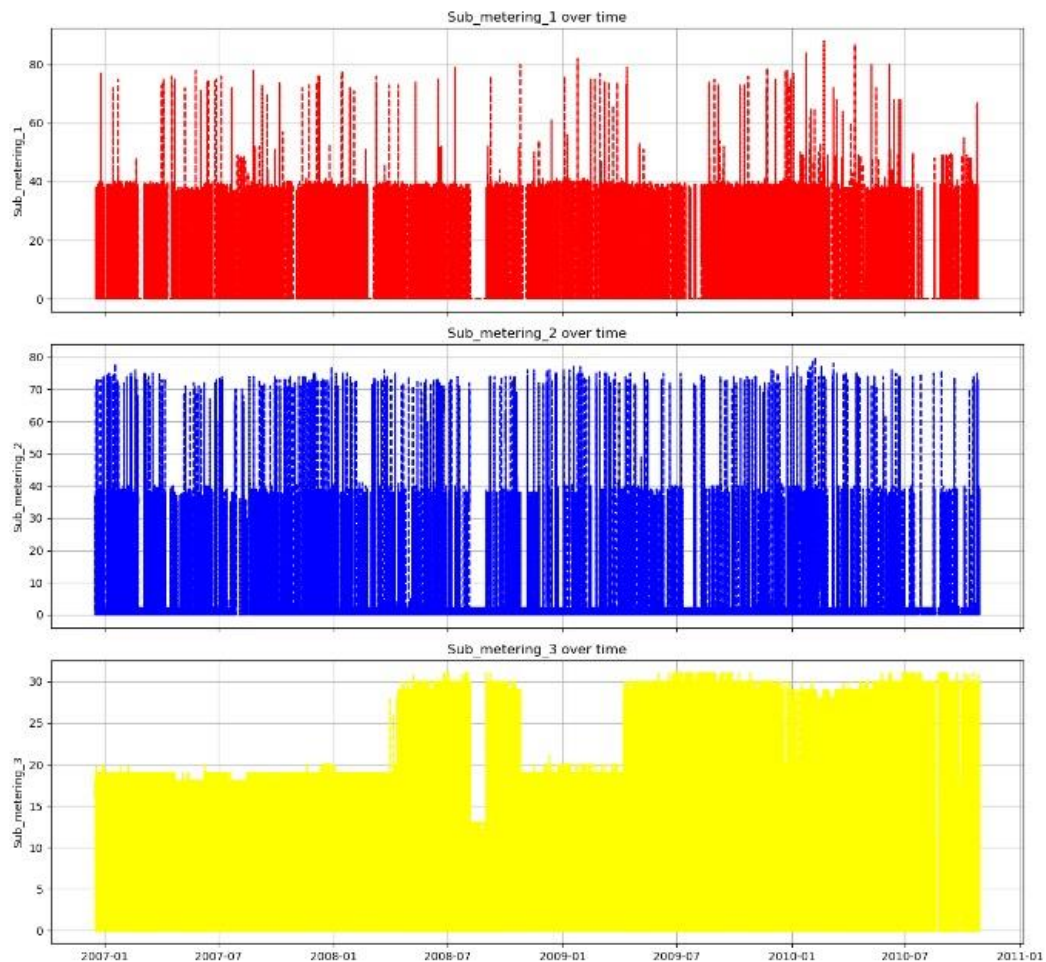


Figure 9: Time-series plots for Sub_metering_1 (kitchen), Sub_metering_2 (laundry), and Sub_metering_3 (HVAC).

A.4 Code Snippet – Derived Feature Calculation and Normalization

- The snippet below highlights how the `Other equipments_energy` feature was computed and how feature scaling was applied using MinMax normalization. This step is critical to ensure numerical stability during model training.

```
# Compute Other Equipments Energy (Residual Consumption)
df['Other equipments_energy'] = (
    df['Global_active_power'] * 1000 / 60
    - df['Sub_metering_1']
    - df['Sub_metering_2']
    - df['Sub_metering_3']
)

# Normalize input features
scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(df[features_imp])
```

- The forecasting model was built using the Sequential API in TensorFlow/Keras. The architecture consists of stacked LSTM layers followed by multiple Dense (fully connected) layers. The final Dense layer outputs 30 values, each corresponding to the forecasted value for one of the 30 future days.

```
model = Sequential([
    LSTM(64, return_sequences=True),
    LSTM(32, return_sequences=True),
    LSTM(32),
    Dense(64, activation='relu'),
    Dense(64, activation='relu'),
    Dense(64, activation='relu'),
    Dense(30) # Output for 30-day forecast
])
```